



Java Threads and Concurrency

Summary

A **Thread** is the smallest unit of execution within a process. Multithreading allows a program to run multiple threads concurrently, significantly improving performance and responsiveness, especially in applications that handle complex tasks or multiple user requests (like web servers).

1. Thread Basics

- **Concurrency:** The ability to execute multiple threads at the same time (or rapidly switch between them on a single core) to maximize resource utilization.
- **CPU Time:** Threads share the same memory space as their parent process but run independently, scheduled by the Operating System.

2. Creating Threads (The Two Contracts)

There are two primary ways to define the code that a new thread will execute. Both methods require implementing the `run()` method, which contains the logic for the thread's execution.

Method A: Implementing the Runnable Interface (Preferred)

This method is generally preferred because Java classes can only inherit from one parent class, but they can implement many interfaces. Using `Runnable` preserves single inheritance.

1. **Implement Runnable:** Create a class that implements the `Runnable` interface and provides an implementation for the `run()` method.
2. **Use Thread:** Pass an instance of your `Runnable` class to a new `Thread` object.

Code Example (Runnable):

```
class Printer implements Runnable {  
    // Defines the code the thread will execute  
    @Override  
    public void run() {  
        System.out.println("Runnable thread is running.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Create the thread, passing the Runnable task  
        Thread t = new Thread(new Printer());  
    }  
}
```

```

        // Start the concurrent execution
        t.start();
    }
}

```

Method B: Extending the Thread Class

This is simpler for small, self-contained tasks, but it uses up your single inheritance slot, as Java only supports single class inheritance.

1. **Extend Thread:** Create a class that extends Thread and overrides the run() method.

Code Example (Extends Thread):

```

class Worker extends Thread {
    // Overrides the Thread's run method
    @Override
    public void run() {
        System.out.println("Thread subclass is running.");
    }
}

public class Main {
    public static void main(String[] args) {
        // Create an instance of the subclass
        Worker t = new Worker();

        // Start the concurrent execution
        t.start();
    }
}

```

3. Key Thread Lifecycle Methods

Method	Purpose	Notes
start()	Starts the thread. It calls the thread's run() method in a separate execution path.	MUST be called to execute code concurrently. Calling run() directly runs code on the current thread.
run()	Contains the actual logic	This method is called by

	that the thread will execute.	the JVM <i>after</i> you call start().
sleep(ms)	Pauses the currently executing thread for a specified number of milliseconds.	This is a static method and throws an InterruptedException, requiring a try-catch block.
join()	Waits for the thread to die (finish execution) before the main thread can continue.	Used to ensure tasks are completed in a specific order.

Lifecycle Example

```

public class ThreadLifecycleExample {
    public static void main(String[] args) {
        // 1. Create a Runnable task using a lambda function
        Runnable task = () -> {
            try {
                System.out.println("Thread running. Going to sleep for 500ms...");
                Thread.sleep(500); // Pause for half a second
                System.out.println("Thread awake and finished.");
            } catch (InterruptedException e) {
                // Handle interruption during sleep
                Thread.currentThread().interrupt();
            }
        };

        // 2. Start the thread
        Thread t1 = new Thread(task);
        t1.start(); // Calls run() concurrently

        // 3. Wait for the thread to finish
        try {
            t1.join(); // Main thread waits here until t1 is done
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("Main thread continues and exits.");
    }
}

```

